

Deployment Guide

This guide explains how to run and deploy the "Operational Ocean Services Knowledge Repository" application. It includes verified local run steps, configuration notes, and recommended hosting options.

Quick verified run (local)

- Requirements: Node.js 20+ (project `engines.node` >= 20.0.0).
- From the repository root run:

```
npm install
npm start
```

- The server listens on `process.env.PORT || 3000`. After startup the app prints:

```
Operational Ocean Services Knowledge Repository API
Server running locally at: http://localhost:3000
```

- While running the application locally you can verify:
 - Frontend: <http://localhost:3000/>
 - Categories API: <http://localhost:3000/api/categories>

Using the included portable Node on Windows

If you don't have Node installed system-wide, the repository includes a portable Node build under `node\_portable/node-v20.12.0-win-x64`.

Open a PowerShell prompt and run:

```
.\node_portable\node-v20.12.0-win-x64\nodevars.bat
npm install
npm start
```

This will set up the environment to use the bundled Node and then install and start the app.

Database configuration

- The app supports two modes:
 - MySQL (production): enabled when `DB\_HOST` and `DB\_NAME` are set in the environment. The server logs: `[Database] Mode: MySQL Server (...)` and uses `src/db/mysql.js`.
 - Local JSON fallback (default): if MySQL vars are not provided the app uses the JSON file DB in `data/db\_fallback` and logs: `[Database] Mode: Local JSON File-Based Database Fallback`.

- Recommended environment variables when using MySQL:

- `DB\_HOST` (required)
- `DB\_NAME` (required)
- `DB\_USER` (optional)
- `DB\_PASS` (optional)
- `DB\_PORT` (optional, default 3306)
- `PORT` (optional, to override the default 3000)

Set variables for example (Linux/macOS):

```
export DB_HOST=your-db-host
export DB_NAME=your_db
export DB_USER=your_user
export DB_PASS=your_pass
export PORT=8080
npm install
npm start
```

On Windows PowerShell:

```
$env:DB_HOST='your-db-host'
```

```
$env:DB_NAME='your_db'
$env:DB_USER='your_user'
$env:DB_PASS='your_pass'
$env:PORT='8080'
npm install
npm start
```

Files and build

- Ensure the following are present in the repo and committed:
 - `server.js` (entry)
 - `public/` (frontend static assets)
 - `src/` (API routes and DB layer)
 - `package.json` and `package-lock.json`
 - `data/db_fallback/` if you plan to use the local JSON DB

Recommended hosting options

- Render, Railway, Heroku, and similar Node-friendly PaaS providers work well. Use `npm install` as the build step and `npm start` as the start command.

Example Render settings:

- Environment: `Node`
- Build command: `npm install`
- Start command: `npm start`

Production considerations

- Use a process manager (PM2, systemd) when deploying to a VM or bare server.
- Serve the app behind a reverse proxy (Nginx) and terminate HTTPS at the proxy.
- If using MySQL, ensure secure credentials and network access only from the application server.
- Persist uploaded files (the `uploads/` folder) on shared or backed-up storage if multiple instances are used.

Verified notes from local run

- I ran the app locally (Windows) with `npm install` and `npm start`.
- The server started successfully and returned the frontend and the categories JSON:
 - `http://localhost:3000/` — frontend HTML served
 - `http://localhost:3000/api/categories` — returned sample categories JSON
- The app used the JSON fallback DB on this machine (log: `Mode: Local JSON File-Based Database Fallback`).

Troubleshooting

- If `npm start` fails, inspect the terminal logs for the specific error.
- If you see DB connection errors, verify `DB_HOST` and `DB_NAME` environment variables for MySQL mode.
- If uploaded files are missing, confirm the `uploads/` folder exists and is writable. The app creates necessary `uploads` subfolders on startup.

If you'd like, I can:

- create a small systemd service or PM2 configuration for production, or
- prepare a Dockerfile and a docker-compose.yml to containerize the app. Which would you prefer?